Projet académique

SMART CANTEEN

Gestion intelligente des files d'attente

RAPPORT DE RÉALISATION

Conception, développement et déploiement de l'application

Matière	PGL
Encadrante	Asmaa Retbi
Étudiant	Isaac DIN Kaougahi
Étudiant	Aboubakar Abdelaziz
Année universitaire	2026–2027
Version	1.0

Table des matières

1 Présentation du projet **3**
1.1 Objet du document . . . 3
1.2 Contexte et problématique . . . 3
1.3 Résultat obtenu . . . 3
1.4 Objectifs . . . 3
1.5 Équipe et démarche . . . 3

2 Conception **4**
2.1 Acteurs et cas d'utilisation . . . 4
2.2 Modèle de données . . . 4
2.3 Règles de gestion . . . 5
2.4 Écarts et ajouts par rapport à la conception UML . . . 6

3 Architecture technique **6**
3.1 Vue d'ensemble . . . 7
3.2 Choix techniques . . . 7
3.3 Organisation du code . . . 8

4 Développement du backend **8**
4.1 Routes par ressource . . . 8
4.2 Réservation sans surréservation . . . 9
4.3 Ticket et contrôle d'accès . . . 9
4.4 File d'attente et notifications . . . 9
4.5 Tâches planifiées et heure du Maroc . . . 9
4.6 Base de données . . . 9

5 Développement du frontend **10**
5.1 Parcours par rôle . . . 10
5.2 Scan du QR code à la caméra . . . 10
5.3 Gestion des menus . . . 10
5.4 Organisation et outils . . . 11

6 Interface et identité visuelle **11**
6.1 Identité . . . 11
6.2 Évolution du design . . . 11
6.3 Principes retenus . . . 12

7 Sécurité, qualité et tests **12**
7.1 Comptes et mots de passe . . . 12
7.2 Faille corrigée . . . 12
7.3 Autres protections . . . 12
7.4 Tests . . . 13

8 Déploiement **13**
8.1 Infrastructure . . . 14
8.2 Étapes suivies . . . 14
8.3 Difficultés rencontrées . . . 15
8.4 Limites de l'hébergement gratuit . . . 15

9 Bilan et perspectives **15**
9.1 Ce qui reste à faire 16
9.2 Ce que le projet nous a appris 16

1 Présentation du projet

1.1 Objet du document

Ce document retrace la réalisation de l'application **Smart Canteen**, de la conception jusqu'à la mise en ligne. Il complète le cahier des charges (v2.0), la conception UML et la planification agile, et décrit ce qui a réellement été construit, les choix faits en chemin et ce qui reste à faire.

1.2 Contexte et problématique

Aux heures de repas, la cantine universitaire accueille un grand nombre d'étudiants en peu de temps. Les étudiants attendent physiquement dans une file, sans connaître leur position ni le temps restant, et la cantine ne peut pas anticiper la demande en repas.

1.3 Résultat obtenu

Smart Canteen est une application web qui permet aux étudiants de l'EMI de réserver leur passage à la cantine et d'entrer avec un ticket QR, au lieu d'attendre dans une longue file. Elle est en ligne depuis octobre 2026 à l'adresse **smartcanteen-two.vercel.app**.

1.4 Objectifs

ID	Objectif	Indicateur cible
OBJ-1	Réduire l'attente physique des étudiants ayant réservé	Attente moyenne ≤ 5 min
OBJ-2	Donner une estimation fiable du temps d'attente	Écart médian ≤ 20 %
OBJ-3	Accélérer le contrôle des tickets	Validation ≤ 3 s
OBJ-4	Permettre à la cantine d'anticiper la demande	Quantités par repas connues avant chaque service

Objectifs repris du cahier des charges v2.0, §1.4.

1.5 Équipe et démarche

Projet réalisé dans le cadre du module PGL par Isaac DIN Kaougahi et Aboubakar Abdelaziz, sous la supervision d'Asmaa Retbi, pour l'année universitaire 2026–2027.

Le projet a suivi quatre étapes : le cahier des charges (v2.0), la conception UML, la planification agile en quatre sprints d'une semaine (05/10 → 01/11), puis le développement et la mise en ligne. Les sections suivantes reprennent chacune de ces étapes.

2 Conception

La conception UML (Visual Paradigm) décrit 4 acteurs, 17 cas d'utilisation et 9 entités métier, tous tracés jusqu'aux exigences du cahier des charges v2.0. Le code reprend cette conception, avec quelques ajouts décidés pendant le développement.

2.1 Acteurs et cas d'utilisation

Acteur	Cas d'utilisation
Étudiant	UC-02 consulter le menu, UC-03 les créneaux, UC-04 réserver, UC-05 consulter le ticket QR, UC-06 suivre la file, UC-07 notifications, UC-08 annuler, UC-09 historique
Agent de contrôle	UC-10 vérifier un ticket (scan, ou saisie du numéro si le scan échoue)
Gestionnaire des repas	UC-11 plats, UC-12 repas et menus, UC-13 signaler un repas indisponible, UC-14 commandes à emporter
Administrateur	UC-15 créneaux et paramètres, UC-16 comptes, UC-17 statistiques

UC-01 (s'authentifier) est commun à tous les acteurs.

2.2 Modèle de données

Les 9 entités du diagramme de classes deviennent des tables PostgreSQL. Des tables de liaison traduisent les relations plusieurs-à-plusieurs, et deux tables gardent la trace des événements.

Entité UML	Tables	Rôle
Utilisateur	`utilisateurs`	Compte, rôle, statut, suspension, absences récentes
Plat	`plats`	Catalogue, désactivable
Repas	`repas`, `repas_plats`	Repas composé de plats, quantité disponible et restante
Menu	`menus`, `menu_repas`, `menu_creneaux`	Un menu par date et par service, ses repas et ses créneaux
Créneau	`creneaux`	Plage horaire, capacité maximale et restante
Réservation	`reservations`, `historique_etats`	Mode, numéro de passage, état, historique des changements d'état
Ticket	`tickets`, `scan_logs`	Jeton QR, validité, journal des scans
Notification	`notifications`	Message, lu ou non
Paramètre	`parametres`	Valeurs réglables sans redéploiement (EF-38)

2.3 Règles de gestion

Les règles du §6 du cahier des charges sont appliquées dans le code, avec ses valeurs par défaut :

- **RG-01 :** une seule réservation active par étudiant, par service et par jour.
- **Fenêtre de réservation :** ouverte 2 jours à l'avance, fermée 30 minutes avant le créneau.
- **RG-02 :** un ticket utilisé, annulé ou expiré n'est plus jamais valable.
- **RG-03 :** numéro de passage unique par jour, file ordonnée par créneau puis par numéro.
- **RG-05/RG-06 :** une réservation non scannée passe « absente » 15 minutes après la fin du créneau ; le ticket reste valable de 10 minutes avant le créneau à 15 minutes après.
- **RG-07 :** 3 absences entraînent 7 jours de suspension. Le compteur repart à zéro après chaque suspension : c'est une simplification de la fenêtre glissante de 30 jours prévue.
- **RG-08 :** attente estimée = personnes devant × durée moyenne de service (45 s par défaut).
- **RG-10 :** un repas signalé indisponible prévient tous les étudiants concernés.
- **RG-11 :** une annulation tardive compte comme une absence.

2.4 Écarts et ajouts par rapport à la conception UML

Point	Conception UML	Application livrée	Raison
Services	Déjeuner, Dîner	Petit-déjeuner, Déjeuner, Dîner	La cantine sert aussi le petit-déjeuner
Créneaux d'un menu	Tous les créneaux de la journée	Le gestionnaire choisit les créneaux de chaque menu	L'étudiant ne réserve que sur les horaires où le repas est servi
Comptes (UC-16)	Comptes du personnel	Tous les comptes, étudiants compris, créés par l'administrateur	Pas d'inscription libre (EF-01) en attendant le SSO
Mot de passe	Non détaillé	Chacun change le sien depuis son profil ; l'administrateur peut le réinitialiser	Le compte est créé avec un mot de passe provisoire
Prix indicatif	Attribut de Repas	Plus saisi par le gestionnaire (vaut 0)	Le règlement reste hors de l'application (§1.5)
Suppression	Non prévue	Créneau, plat, repas et menu supprimables sous conditions	Corriger une erreur de saisie sans casser l'historique

3 Architecture technique

L'application suit l'architecture client-serveur en trois couches demandée par TECH-01 : une application React dans le navigateur, une API REST en JSON, et une base PostgreSQL. Le code tient en un seul dépôt Git avec deux dossiers, `backend/` (environ 2 000 lignes de JavaScript et 225 lignes de SQL) et `frontend/` (environ 3 300 lignes de JSX et 2 500 lignes de CSS).

3.1 Vue d'ensemble

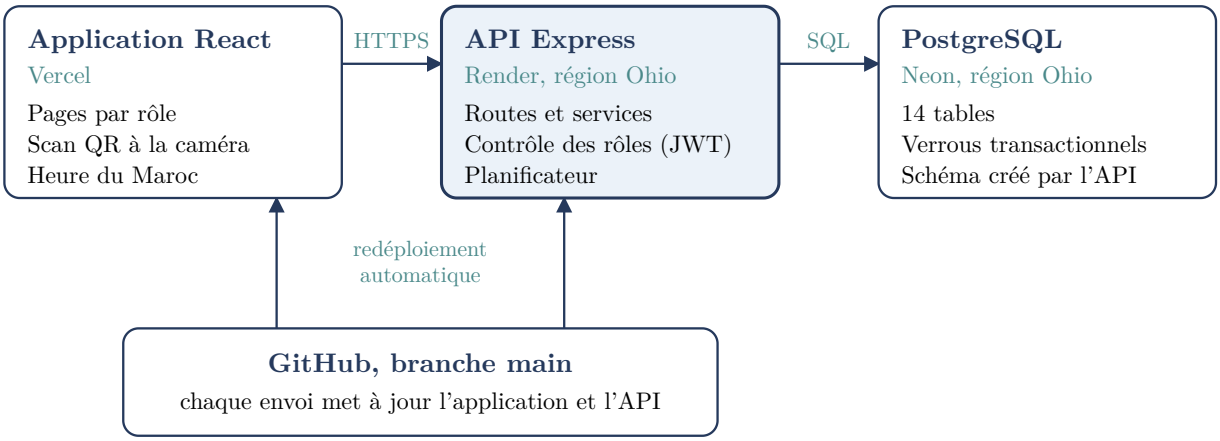


Figure 1 — Architecture : trois couches hébergées séparément, redéployées depuis GitHub.

L'application appelle l'API en HTTPS ; seule l'API parle à la base. Un envoi sur GitHub met à jour l'application et l'API, et l'API crée ou fait évoluer le schéma de la base à son démarrage.

3.2 Choix techniques

Couche	Choix	Pourquoi
Base de données	PostgreSQL, SQL écrit à la main (driver `pg`)	Transactions, contraintes et verrous `SELECT … FOR UPDATE` contre la double réservation (TECH-04)
API	Node.js et Express 4, validation des entrées avec `zod`	Proposé par TECH-03, léger, une route par ressource
Authentification	JWT, mots de passe hachés avec bcrypt	Sessions sans état côté serveur
Ticket	Jeton JWT signé (HMAC-SHA256) affiché en QR code	Infalsifiable, sans donnée personnelle en clair (TECH-06)
Application	React 19, Vite 8, React Router 7	Proposé par TECH-02, rapide à développer, mobile d'abord
QR code	`qrcode.react` pour l'affichage ; `jsqr` et le détecteur du navigateur pour le scan	Fonctionne sur Android comme sur iPhone
Hébergement	Neon (base), Render (API), Vercel (application)	Offres gratuites, HTTPS fourni, redéploiement à chaque envoi sur GitHub

Prisma avait d'abord été envisagé comme ORM, puis écarté : son moteur est téléchargé à l'installation et le réseau du poste de développement le bloquait. Le SQL écrit à la main donne

en plus un contrôle direct sur les verrous.

3.3 Organisation du code

```
smartcanteen/
├── README.md, render.yaml, docs/DEPLOIEMENT.md
├── backend/
│   ├── db/          schema.sql, migrations/, setup.js, seed.js
│   └── src/         server.js, app.js, routes/, services/, middleware/, lib/
└── frontend/
    └── src/         App.jsx, api/, context/, lib/,
                     components/ (ui, layout, marque),
                     pages/ (etudiant, agent, gestionnaire, administrateur),
                     styles/ (thème, base, composants, pages)
```

Côté API, les **routes** reçoivent et valident les requêtes, les **services** portent les règles métier, et `lib/` regroupe l'accès à la base, l'heure du Maroc et les paramètres. Côté application, chaque rôle a son dossier de pages, et les composants partagés sont rangés à part.

4 Développement du backend

L'API expose 38 points d'entrée répartis en 10 fichiers de routes, tous sous `/api` et protégés par rôle. Le cœur métier, la réservation, tient dans une seule transaction SQL qui ne peut jamais dépasser la capacité d'un créneau.

4.1 Routes par ressource

Ressource	Principales actions	Rôle
`/auth`	Connexion, profil, changement de mot de passe	Tous
`/menus`, `/creneaux`	Consulter ; publier, dupliquer ou retirer un menu ; créer, modifier ou supprimer un créneau	Étudiant, gestionnaire, administrateur
`/plats`, `/repas`	Créer, modifier, supprimer ; signaler un repas indisponible	Gestionnaire
`/reservations`	Réserver, mes réservations, ticket, position dans la file, annuler	Étudiant
`/tickets/scan`	Vérifier un ticket par son jeton QR ou son numéro	Agent
`/commandes`	Suivre et faire avancer les commandes à emporter	Gestionnaire
`/notifications`	Lister, marquer comme lue	Étudiant
`/admin`	Comptes, mots de passe, paramètres, statistiques	Administrateur

Un middleware vérifie le jeton JWT puis le rôle de chaque requête. Les entrées sont validées avec `zod`, et chaque erreur renvoie un message clair en français (ENF-21).

4.2 Réservation sans surréservation

La réservation (UC-04) suit le diagramme de séquence de la conception, dans une seule transaction :

1. Vérifier la fenêtre : au plus 2 jours à l'avance, au plus tard 30 minutes avant le créneau.
2. Vérifier que le repas est servi sur ce créneau, c'est-à-dire que le gestionnaire a lié ce créneau à son menu.
3. Vérifier RG-01 : aucune autre réservation active pour ce service et ce jour.
4. Verrouiller le créneau et le repas (`SELECT … FOR UPDATE`) : deux étudiants simultanés ne peuvent pas prendre la dernière place.
5. Décrémenter la capacité et la quantité, attribuer le numéro de passage, créer le ticket.
6. Valider la transaction ; en cas de refus, tout est annulé et l'étudiant reçoit le motif, avec des créneaux alternatifs si le sien est complet (EF-14).

4.3 Ticket et contrôle d'accès

Le QR code contient un jeton JWT signé avec l'identifiant de la réservation et une date d'expiration, sans donnée personnelle. Le ticket est valable de 10 minutes avant le créneau à 15 minutes après sa fin. Au scan, l'API vérifie la signature, la fenêtre et l'état, puis renvoie un verdict : valide, déjà utilisé, annulé, trop tôt ou expiré. Un ticket validé passe « utilisé » dès le premier scan, ce qui empêche de le réutiliser (RG-02), et chaque scan est journalisé.

4.4 File d'attente et notifications

- La position est le nombre de personnes non servies devant l'étudiant, plus un (RG-03).
- L'attente estimée multiplie cette position par la durée moyenne des 15 derniers passages, 45 secondes par défaut, plafonnée à 5 minutes par personne (RG-08).
- L'écran du ticket se met à jour toutes les 15 secondes, la solution de repli prévue par TECH-05.
- L'étudiant est notifié quand il reste 10 personnes devant lui, et quand un repas qu'il a réservé devient indisponible (RG-10).

4.5 Tâches planifiées et heure du Maroc

Un planificateur intégré à l'API passe toutes les minutes et marque « absentes » les réservations non scannées 15 minutes après la fin de leur créneau (TECH-09). Les absences répétées déclenchent la suspension prévue par RG-07.

Les heures sont stockées en UTC et affichées à l'heure du Maroc (TECH-13). Le Maroc a changé de régime horaire ; la base de fuseaux de notre PostgreSQL local était périmée et décalait les heures d'une heure. Tous les calculs de fuseau passent donc par Node.js, dont la base est à jour, et l'API fournit son décalage aux écrans (`/api/heure`) pour qu'un téléphone mal réglé n'affiche pas une mauvaise heure.

4.6 Base de données

Le schéma est créé par `schema.sql`, puis les évolutions passent par des migrations rejouables. Le script `npm run setup` crée le schéma s'il n'existe pas, applique les migrations et

charge les données de démonstration dans une base vide. Il s'exécute à chaque démarrage de l'API en ligne.

5 Développement du frontend

L'application est une seule page React : après la connexion, chaque rôle voit sa propre navigation et sa propre page d'accueil. Elle est pensée d'abord pour le téléphone (barre de navigation en bas), avec une barre latérale sur ordinateur.

5.1 Parcours par rôle

Rôle	Pages	Ce qu'on y fait
Étudiant	Menu, Mes tickets, Ticket, Notifications	Voir le menu du jour par service, réserver un créneau sur place ou à emporter, afficher son ticket QR, suivre sa position et l'attente, annuler
Agent de contrôle	Contrôle	Scanner un ticket à la caméra, ou saisir le numéro de passage si le scan échoue ; verdict en grand, vert ou rouge
Gestionnaire des repas	Plats, Menus, Commandes	Gérer le catalogue ; composer un repas puis le publier sur une date, un service et des créneaux ; faire avancer les commandes à emporter
Administrateur	Créneaux, Comptes, Statistiques	Créer les créneaux du jour et leur capacité, créer les comptes, consulter l'affluence, le taux d'absence et les repas les plus demandés

Tous les rôles ont un profil (nom, email, rôle) où changer son mot de passe et se déconnecter.

5.2 Scan du QR code à la caméra

Le lecteur utilise le détecteur de QR code intégré au navigateur quand il existe (Chrome, Android), et la bibliothèque `jsqr` sinon (Safari, iPhone). Un même ticket resté devant la caméra n'est pas renvoyé en boucle, et le verdict reste affiché le temps d'être lu. Les navigateurs n'ouvrent la caméra que sur une page en HTTPS : pour tester sur téléphone avant la mise en ligne, nous avons ajouté un mode `npm run dev:mobile` qui sert l'application en HTTPS sur le réseau local.

5.3 Gestion des menus

Le gestionnaire travaille en deux étapes sur la même page : composer un repas à partir des plats, puis publier le menu en choisissant la date, le service et les créneaux où il est servi. Si le menu existe déjà, ses créneaux actuels sont précochés et la publication les remplace. Chaque action affiche son résultat juste sous son bouton : un premier problème était qu'une publication échouait sans aucun message.

5.4 Organisation et outils

- Un seul fichier `api/client.js` regroupe tous les appels à l'API ; l'adresse de l'API se règle par `VITE_API_URL`.
- La session (jeton JWT et utilisateur) est partagée par un contexte React.
- Les composants partagés (boutons, fenêtres, badges, icônes, fenêtre de confirmation) évitent de dupliquer le code entre les pages.
- Les dates et heures passent toutes par `lib/format.js`, qui affiche l'heure du Maroc fournie par le serveur.
- Le code est vérifié par l'outil d'analyse `oxlint`.

6 Interface et identité visuelle

L'interface finale est un thème clair : fond blanc porcelaine, accent bleu roi, texte bleu nuit, et un ticket couleur laiton qui ressort sur tout le reste. Elle a été retravaillée plusieurs fois, au fil des retours sur les versions précédentes.

6.1 Identité

- **Logo :** une toque de chef entre une cuillère et une fourchette, dessinée en SVG, reprise dans l'icône de l'onglet.
- **Ancrage local :** un motif de zellige discret en fond du panneau de connexion et du ticket, et l'emblème de l'EMI en filigrane sur la page de connexion.
- **Accroche :** « Votre repas, sans faire une longue queue. »
- **Typographies :** Unbounded pour les titres et les numéros, Instrument Sans pour le texte.

6.2 Évolution du design

Version	Direction	Ce qui a changé ensuite
1	Interface simple, fonctionnelle	Jugée trop simple
2	Thèmes sombres successifs, puis « thé à la menthe » : menthe et laiton sur fond vert pétrole, panneaux de verre, fond animé	Retour à un fond clair demandé
3	Thème blanc, accent menthe	Accent passé au bleu
4 (actuelle)	Thème blanc, accent bleu roi	—

Le vert reste réservé aux états de réussite (réservation confirmée, ticket valide) pour garder son sens, et le rouge aux erreurs.

6.3 Principes retenus

- **Mobile d'abord :** l'étudiant et l'agent utilisent surtout leur téléphone ; la navigation est en bas de l'écran, les zones à toucher sont larges.
- **Le ticket comme objet :** numéro de passage en très grand, QR code bien contrasté pour un scan rapide, et la file d'attente dessinée (le comptoir, les personnes devant, puis « vous »).
- **Un retour visible pour chaque action :** chaque formulaire affiche son résultat juste sous son bouton, et chaque suppression demande une confirmation.
- **Accessibilité :** les animations sont coupées quand le téléphone le demande (réduction des mouvements), et les icônes seules portent un libellé lu par les lecteurs d'écran.
- **Code :** les couleurs principales sont définies une seule fois dans `styles/theme.css`, ce qui a rendu les changements de palette rapides.

7 Sécurité, qualité et tests

Les comptes ne peuvent être créés que par l'administrateur, chaque action de l'API vérifie le rôle de la personne connectée, et les fonctions sensibles ont été testées de bout en bout avant chaque mise en ligne.

7.1 Comptes et mots de passe

- **Pas d'inscription libre (EF-01) :** l'administrateur crée tous les comptes, étudiants compris, avec un mot de passe provisoire de 8 caractères minimum. L'application peut le générer, sans caractères ambigus (0/O, 1/l), pour qu'il se dicte sans erreur.
- Chacun change son mot de passe depuis son profil, en donnant l'actuel. En cas d'oubli, l'administrateur en attribue un nouveau.
- Les mots de passe sont hachés avec bcrypt ; un échec de connexion ne dit pas si c'est l'email ou le mot de passe qui est faux.
- Un administrateur ne peut pas suspendre son propre compte, et un compte suspendu ne peut plus se connecter.
- Les identifiants de démonstration ne sont plus affichés sur la page de connexion.

7.2 Faille corrigée

La première version gardait une route d'inscription publique, prévue pour les tests. Elle acceptait le rôle demandé dans la requête : n'importe qui pouvait donc se créer un compte administrateur. Nous avons supprimé cette route et vérifié sur la base en ligne qu'aucun compte n'avait été créé par ce moyen.

7.3 Autres protections

- Le secret qui signe les sessions et les QR codes est généré par Render ; l'API refuse de démarrer en production avec la valeur d'exemple.
- L'API peut limiter les appels à l'adresse de l'application (variable `CORS_ORIGIN`), mais elle n'est pas encore renseignée sur Render : l'API en ligne accepte pour l'instant les ap-

pels de toute origine. Un étudiant ne peut lire que ses propres réservations (ENF-37).

- Les fichiers `.env` contenant les secrets ne sont jamais envoyés sur GitHub.
- La suppression est encadrée : un créneau ou un repas déjà réservé, même une fois annulé, est conservé pour que l'historique et les statistiques restent justes ; un menu ne peut pas être retiré tant que des étudiants l'ont réservé ; un plat est désactivé plutôt qu'effacé.

7.4 Tests

Chaque fonctionnalité a été testée sur une base de test créée pour l'occasion, puis supprimée, sans toucher aux données réelles.

Domaine	**Ce qui est vérifié**	**Résultat**
Comptes et mots de passe	Inscription publique fermée, création par l'administrateur, changement et réinitialisation du mot de passe, droits par rôle, suspension	17 tests sur 17
Suppressions	Suppression de chaque élément, refus quand il est réservé, droits par rôle	19 tests sur 19
Interface	Chaque page des quatre rôles s'ouvre sans erreur	10 pages sur 10

Ces tests ont aussi permis de corriger deux bugs : un blocage de la base pendant la réservation (une notification écrite hors de la transaction en cours attendait le verrou posé par cette même transaction), et l'affichage des heures décalé d'une heure.

Les tests de charge (k6 ou JMeter) et l'intégration continue prévus par TECH-10 et TECH-11 n'ont pas encore été mis en place, et les tests ci-dessus sont des scripts lancés à la main.

8 Déploiement

L'application est en ligne gratuitement sur trois services, alimentés par le dépôt GitHub : chaque envoi sur la branche `main` redéploie l'API et l'application en moins d'une minute.

8.1 Infrastructure

Brique	Service	Adresse	Configuration
Code	GitHub	github.com/Aboubakar235/smartcanteen	Dépôt public, branche `main`
Base de données	Neon (PostgreSQL)	Région AWS US East 2 (Ohio)	Chaîne de connexion fournie à Render, jamais dans le code
API	Render	smartcanteen-api-pk1y.onrender.com	`render.yaml` : dossier `backend`, région Ohio, `npm run setup && npm start`
Application	Vercel	smartcanteen-two.vercel.app	Dossier `frontend`, variable `VITE_API_URL`, `vercel.json` pour les routes

8.2 Étapes suivies

1. Préparer le code : adresse de l'API réglable par variable d'environnement, script `npm run setup` qui crée la base au premier démarrage, refus de démarrer sans secret JWT, fichiers de configuration Render et Vercel.
2. Envoyer le code sur GitHub, avec un jeton d'accès personnel ; les fichiers `.env` restent exclus.
3. Créer la base sur Neon et récupérer sa chaîne de connexion.
4. Créer l'API sur Render à partir de `render.yaml` (Blueprint), en collant la chaîne Neon ; Render génère le secret JWT et l'API crée les tables et la démo au démarrage.
5. Créer l'application sur Vercel, dossier `frontend`, avec l'adresse de l'API dans `VITE_API_URL`.
6. Tester sur plusieurs appareils, dont le scan à la caméra sur téléphone, possible grâce au HTTPS.

Le guide pas à pas est dans `docs/DEPLOIEMENT.md`.

8.3 Difficultés rencontrées

Problème	Solution
Le téléphone de l'agent ne pouvait pas ouvrir la caméra sur l'adresse locale en HTTP	Mode HTTPS local (`npm run dev:mobile`) avec un certificat de développement
Le Wi-Fi de l'école bloquait l'accès du téléphone à l'ordinateur, puis les tunnels (Cloudflare, localhost.run) étaient bloqués ou trop lents	Abandon des tunnels au profit d'un vrai déploiement en ligne
L'API et la base dans deux régions éloignées auraient ralenti chaque requête	API placée dans la même région que la base (Ohio)
Heures affichées avec une heure de décalage	Calculs de fuseau dans Node.js et heure du Maroc fournie par l'API
Route d'inscription publique exposée en ligne	Route supprimée, puis redéploiement

8.4 Limites de l'hébergement gratuit

- L'API Render s'endort après 15 minutes sans visite ; la première ouverture suivante prend environ 50 secondes, et le marquage automatique des absences ne tourne que lorsque l'API est réveillée.
- Les comptes de démonstration existent sur la base en ligne : leurs mots de passe doivent être changés, et `SEED_DEMO` passé à `false`, avant une utilisation réelle.
- La conteneurisation Docker prévue par TECH-10 n'a pas été nécessaire : Render et Vercel construisent l'application directement depuis le dépôt.

9 Bilan et perspectives

Le MVP réalise les neuf user stories du backlog et l'essentiel des 17 cas d'utilisation ; il est utilisable en ligne sur ordinateur comme sur téléphone. Il manque surtout des éléments d'infrastructure (tests de charge, intégration continue, notifications push) et trois écrans secondaires.

9.1 Ce qui reste à faire

Point	Exigence	État
Connexion par le compte universitaire (SSO)	TECH-07	Remplacée par des comptes créés par l'administrateur
Notifications push sur le téléphone	TECH-08	Notifications visibles dans l'application seulement
Mise à jour de la file en temps réel (WebSocket)	TECH-05	Rafraîchissement toutes les 15 secondes, le repli prévu
Application installable hors ligne (PWA)	TECH-02	Site adaptatif, non installable
Tests de charge et intégration continue	TECH-10, TECH-11	Tests de bout en bout lancés à la main
Modifier une réservation	EF-16	Non réalisé : l'étudiant annule puis réserve à nouveau
Réglage des paramètres (délais, capacités)	EF-38	Disponible dans l'API, pas encore dans l'interface
Duplication d'un menu	EF-09	Disponible dans l'API, pas encore dans l'interface
Restriction des appels à l'adresse de l'application	—	Variable `CORS_ORIGIN` à renseigner sur Render

9.2 Ce que le projet nous a appris

- Une conception détaillée en amont (cas d'utilisation, modèle de données, règles numérotées) a rendu le développement plus direct : chaque route et chaque règle se rattache à une exigence.
- La concurrence se traite dans la base : un verrou au bon endroit suffit à garantir qu'un créneau ne dépasse jamais sa capacité.
- Le fuseau horaire est un vrai sujet au Maroc, à traiter dès le début.
- Tester sur un vrai téléphone, en conditions réelles, a révélé des contraintes invisibles sur ordinateur (HTTPS pour la caméra, réseau de l'école).
- La sécurité se vérifie : une route de test oubliée suffisait à créer un compte administrateur.